Ders 5 Genişletilmiş Notu: Facade, Flyweight, Proxy, Chain of Responsibility ve Command Kalıpları

Ders 5 Genişletilmiş Notu: Facade, Flyweight, Proxy, Chain of Responsibility ve Command Kalıpları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Facade Kalıbı
- 2. Flyweight Kalıbı: Amaç, Örnek ve Sorun
- 3. Flyweight Kalıp Yapısı ve İçsel/Dışsal Durum
- 4. Flyweight Gerçekleme Ayrıntıları ve Kullanılabileceği Anlar
- 5. Flyweight Uygulama Örneği: Kimya Fabrikası
- 6. Proxy Kalıbı
- 7. Proxy Uygulama Örneği: İzlediğin Kadar Öde Sunucusu
- 8. Davranışsal Kalıplara Geçiş: Chain of Responsibility
- 9. Chain of Responsibility Uygulama Örneği: Reklam Yönetim Yazılımı
- 10. Command Kalıbı
- 11. Command Uygulama Örneği: Akıllı Ev Kumandası
- Örnek Sorular
- Dersin Toparlaması

1. Facade Kalıbı

Bir önceki haftada yapısal kalıplardan Composite ve Decorator işlenmişti. Bu ders, yapısal grubun sıradaki kalıbı Facade ile başladı. Hoca, adın İtalyanca kökenli “faça” sözcüğüyle (yüz, sima, surat) akılda tutulabileceğini söyledi: kalıp, bir sistemin dışarıya dönük tek yüzünü oluşturur.

Kalıbın amacı, çeşitli alt sistemlerde bulunan farklı arayüzleri tek, ortak ve daha üst düzey bir arayüzde toplamaktır. Böylece istekçi nesneler, farklı alt sistemlere bağlanıp farklı farklı metotları çalıştırmak zorunda olduklarını bilmeden, gereken bütün metotları tek bir erişim noktasından çağırabilirler.

Slayttaki şemada bir dörtgenin içinde karmaşık bir sınıf tasarımı yer alır. İstemcinin bu sınıflardan bazılarının bazı metotlarını tek tek çağırması yerine bir Facade sınıfı bulunur. Facade’ın metotları, alt sistemdeki ilgili sınıfların ilgili nesnelerinin metotlarını (`M1`, `M2` gibi) çağırır, sonucu alıp geri döndürür. Hoca kalıbın başka bir numarası olmadığını söyledi.

Hoca bunu karmaşıklığı halının altına süpürmeye benzetti: evin hemen toplanması gerektiğinde her şeyi yerli yerine koymak uzun süreceği için ortada ne varsa bir dolaba tıkıştırmak gibi. Çizgi filmlerde ve günlük hayatta sıkça görülen bu davranışın mantığı kalıbın mantığıdır. Hoca bu kalıp için ayrı bir örnek vermeye gerek görmedi.

2. Flyweight Kalıbı: Amaç, Örnek ve Sorun

2.1 Amaç

Flyweight, spordaki sıklet adlarından gelir; tüy sıklet ya da hafif sıklet diye çevrilebilir. Slayttaki amaç şudur (ekrandan, 00:10:00): paylaşım yolu ile çok sayıda basit nesnenin kullanımının desteklenmesi.

2.2 Örnek: Metin Düzenleme Programı

Bir metin düzenleme programı yazılıyor. Hoca durumu daha bütüncül görmek için biraz eski nesil, sabit genişlikli fontlarla ve sabit satır-sütunlardan oluşan bir metin düşünmemizi istedi; sabit genişlikli olsa da farklı fontlar bulunur. Program, font ve diğer biçimlendirme bilgilerine (eğik, koyu, altı çizili vb.) göre metni ekranda ve yazıcıda çok net göstermelidir (excellent rendering). Bu amaçla harfler grafik nesneleri olarak gösterilir; grafikler ne kadar ayrıntılıysa gösterim o kadar iyi olur. Dosya biçimi olarak da metin, sütunlar ve satırlar içinde bulunan harfler olarak ele alınabilir: hangi satırın hangi sütununda hangi harf olduğu tutulur.

2.3 Sorun

Her harf ayrı bir nesne örneği olarak ele alınırsa, küçük bir belgede bile binlerce, hatta on binlerce nesne oluşur. Slayttaki örnek: bu dersin üç sayfalık öneri formunda bile boşluklarla birlikte 5000 kadar harf vardır. Bu kadar çok nesne hem bellekte hem de disk gibi ikincil saklama ortamlarında aşırı yer kaplar. Eski bir sistem varsayıldığı için bellekten tasarruf etmek zorunludur.

2.4 Çözüm Fikri

Metnin her satır ve sütun konumunda harf nesnesi oluşturmak yerine, o harfi simgeleyen tek bir anahtar alan tutulur. Bu anahtar, kullanılan karakter kümesindeki harfleri karşılayacak kadar geniş bir veri tipindendir (bir ya da iki bayt). Hoca bunları bir işaretçi gibi düşünmemizi söyledi. Asıl harfler, yani grafiksel olarak tasarlanmış harfler, her harf için bir tane oluşturulur ve bir havuzda (flyweight pool) toplanır. Her konumdaki anahtar, bizi havuzdaki ilgili nesneye yönlendirir. Böylece harf nesneleri ortak bir havuzda paylaşımlı kullanılır; bu şekilde paylaşılan nesnelere flyweight denir.

Hoca kazancı sayılarla anlattı. Bir milyon harflik bir metin düşünelim:

- Her konumda yalnız anahtar tutulursa: UTF-16 karakter başına 2 bayt ister, yani $10^6 \times 2 = 2 \times 10^6$ bayt; buna havuzdaki harf grafiklerinin kapladığı alan eklenir.
- Bir milyon harfin her biri 1 KB'lık ayrı bir grafik nesnesi olarak saklansaydı: $10^6 \times 1\,\text{KB} \approx 1\,\text{GB}$ gerekirdi.

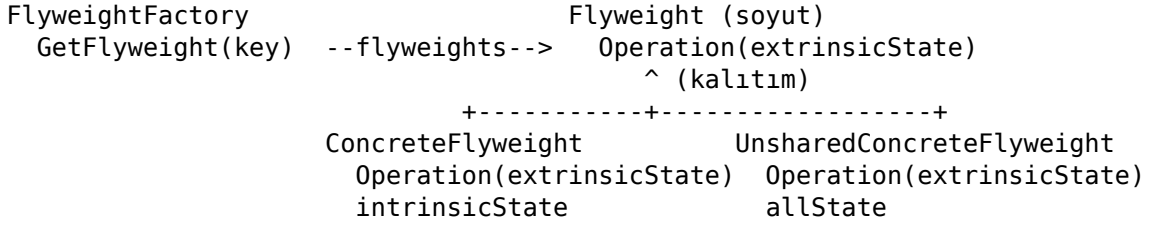
Havuzdaki nesneler ne kadar büyükse, paylaşımla elde edilen tasarruf o kadar artar.

GoF'un önerdiği yapıda satırlar, sütunlar ve karakterler ortak bir kalıtım hiyerarşisi altında glif olarak ele alınır. Hoca bunun bir önceki derste görülen Composite kalıbına benzediğini söyledi. Bir satırın çizilmesi istendiğinde, o satırdaki karakterler için havuzdan ilgili nesne alınır ve ilgili satır ve sütuna çizdirilir.

3. Flyweight Kalıp Yapısı ve İçsel/Dışsal Durum

3.1 Kalıp Yapısı

Slayttaki kalıp yapısı (ekrandan, 00:18:00) metne şöyle dökülebilir:



```
Client --> FlyweightFactory
Client --> ConcreteFlyweight, UnsharedConcreteFlyweight

GetFlyweight(key) notu:
  if (flyweight[key] exists) {
      return existing flyweight;
  } else {
      create new flyweight;
      add it to pool of flyweights;
      return the new flyweight;
  }
```

`FlyweightFactory`, flyweight'lerin yerini tuttuğu daha büyük nesnelerin havuzunu saklar. Bir getter metodu vardır: anahtarı verirsiniz, size o anahtarın simgelediği nesneyi döndürür. Hoca, anahtara karşılık bir nesne yoksa ne yapılacağının senaryoya bağlı olduğunu söyledi: nesne bir yerden yüklenebilir, `null` döndürülebilir ya da ileride görülecek Proxy kalıbıyla, asıl nesne yüklenene kadar onun yerine geçen bir proxy nesnesi döndürülebilir.

Soyut `Flyweight` sınıfının yapması gereken metotlar soyut tanımlanır. Bütün metotları soyut olduğu için bu sınıf Java'da bir arayüz olarak da kodlanabilir. Somut flyweight'ler bundan kalıtımla türetilir. Genelde flyweight'ler paylaşılır, ama paylaşılmayanlar ya da ileride proxy olarak kullanılacak özel flyweight'ler varsa, bunlar kalıtımda kardeş sınıf olarak eklenir.

3.2 İçsel ve Dışsal Durum Bilgisi

Flyweight ve onun simgelediği nesne aslında birlikte bir bütün oluşturur. Bu bütünün değişmeyen ve hep aynı kalan kısmı içsel (intrinsic) durumdur; metin örneğinde harfin grafiğidir. Dışsal (extrinsic) durum ise harfin hangi satır ve sütuna konacağıdır. İçsel durum ortak kullanıldığı için, ne kadar büyükse o kadar tasarruf edilir; dışsal durum ise kullanıldığı yerden flyweight'e verilir.

Slayttaki gerçekleme ayrıntıları (ekrandan, 00:20:00):

- Kalıbın etkili olabilmesi için, paylaşılacak nesnelerin durum bilgisinin büyük bir kısmı çalışma anında hesaplanabilir olmalıdır.
  - Bu sayede flyweight nesnelerinin saklanması gereken durum bilgisi çok azalır ve kalıbın etkinliği artar.
  - Saklanması gereken durum bilgisine içsel (intrinsic), hesaplanabilen ve böylece harici bir nesneden bu nesneye bir metot çağrısı ile aktarılabilen durum bilgisine ise dışsal (extrinsic) adı verilir.
  - Alternatif terminoloji: intrinsic → immutable, extrinsic → mutable. İçsel kısmı oluşturan üyeler `final` olarak tanımlanabilir.
- Her flyweight nesnesinin paylaşılması gerekmeyebilir.

Hoca alternatif terminolojiyi şöyle açıkladı: mutable bilgi, flyweight'in referans edildiği her yer için değişir. Immutable kısım değişmez; belirli bir font için (ör. Consolas ya da Times New Roman) A harfinin resmi sabittir. A harflerinin geçtiği x, y koordinatları ise değişen bilgidir.

> **Ek bilgi (derste anlatılmadı):** Hoca bir yerde (00:22:23) "dışsal bilgi ne kadar fazlaysa o kadar tasarruf" dedi, başka yerlerde (00:19:10, 00:29:27) "içsel bilgi ne kadar büyükse o kadar tasarruf" dedi. İkisi büyük olasılıkla aynı şeyin iki yüzüdür: nesnenin durumundan ne kadar çok kısım dışarı alınıp saklanmadan hesaplanabilirse paylaşılan nesne o kadar sadeleşir; paylaşılan içsel kısım ne kadar büyükse ve ne kadar çok yerde paylaşılırsa kazanç o kadar artar.

3.3 Paylaşılmayan Flyweight'ler ve Sadeleştirme

Her nesnenin paylaşılması gerekmeyebilir. Bu durumda paylaşılmayanlar `UnsharedConcreteFlyweight` gibi bir kardeş sınıf olarak yazılabilir; ya da paylaşılıp paylaşılmama mantığı tek bir `ConcreteFlyweight` sınıfına konabilir. Kardeş sınıfa ihtiyaç yoksa soyut üst sınıf da kaldırılabilir ve fabrika doğrudan `ConcreteFlyweight` döndürür. Hoca böylece kalıbın "daha da hafif sıklette" tasarlanabileceğini söyledi.

Çalışma anındaki nesne ilişkilerinde, bellekte `ConcreteFlyweight` nesneleri bulunur ve fabrika bunları idare eder. Bir istemci flyweight istediğinde önce fabrikaya sorar, sonra fabrikadan aldığı nesneyle iletişim kurar.

4. Flyweight Gerçekleme Ayrıntıları ve Kullanılabileceği Anlar

4.1 Gerçekleme Ayrıntıları

- Fabrika nesnesi isteğe bağlı gibi düşünülebilir, ama hocaya göre isteğe bağlı değildir: anahtarı verip ilgili nesneyi almak her durumda bir havuzla gerçeklenecektir ve o havuzu fabrika olarak düşünmek en mantıklısıdır.
- Flyweight nesneleri Singleton gibi düşünülebilir, ama buna gerek yoktur; fabrika zaten ilgili anahtar için ilgili nesneyi döndürür.
- Erişimi daha sıkı denetlemek isteniyorsa, flyweight nesnesinin işlem metotlarını bir arayüzde tanımlamak iyi olur. Flyweight sınıfı fabrikanın `private` iç sınıfı yapılabilir.
- Nesnenin kendisini döndürmek yerine bir klonunu ya da ona sınırlı erişim açan bir handle'ı göndermek de düşünülebilir. Hoca örnekte buna girmeyeceklerini, isteyenin denemesini söyledi.
- `private` iç sınıf kullanımı ileride Memento kalıbında yapılacaktır. Orada mementolara erişimi kısmak çok daha kritiktir; burada kritik olmadığı için yapılmamıştır.

4.2 Kullanılabileceği Anlar

Hoca derste slaytı düzenleyerek maddeleri toparladı (ekrandan, 00:30:00). Kalıbın kullanılabilmesi için şu koşulların tümünün doğru olması gerekir; aksi halde getirdiği yararlar azalır:

- Bir uygulamanın çok fazla sayıda nesne kullandığı,
- Nesnelerin çok fazla olması nedeniyle saklama masraflarının çok yüksek olduğu,
- Nesnelerin durum bilgisinin içsel ve dışsal olarak iki bölüme ayrılabildiği,
- Dışsal durumları ayrılan nesnelerin birçok grubunun yerini, paylaşılabilecek az sayıda nesnenin alabileceği,
- Uygulamanın nesne kimliğine bağlı olmadığı anlarda.

Hoca üçüncü maddeyi kitabın çevirisinden "durum bilgisinin çoğunun dışsallaştırılabilmesi" diye okudu, sonra daha iyi bir çeviri olarak "nesnenin durum bilgisinin içsel ve dışsal olarak iki bölüme ayrılabilmesi; içsel bilgi ne kadar çoksa ortak kullanım sayesinde elde edilen tasarruf o kadar yüksek olur" biçiminde düzeltti.

4.3 Nesne Kimliği

> **Hocanın vurgusu:** Nesne kimliği burada kritiktir.

Uygulamada her konum için ayrı nesne kullanılsaydı, bunların hepsi farklı nesneler olurdu; bellekte farklı adreslerden başlayarak saklanır ve her birinin ayrı bir kimliği olurdu. Flyweight'te ise bütün nesneler aynı havuzdadır ve konumlarda yalnız onlara işaretçiler tutulur. Aynı harfi gösteren bütün konumlar bizi aynı nesneye götürür; bağımsız bir nesne kimliği yoktur. Kelime işlemci örneğinde bunun hiçbir sakıncası yoktur.

> **Ders sonrası not:** Hoca (00:27:04) Java'nın her nesneyi tanımlamak için bir "universal unique identifier" koyduğunu söyledi. Java nesnelere UUID atamaz; nesne kimliği referansların == ile karşılaştırılmasıyla belirlenir. Hocanın asıl vurgusu, yani farklı nesnelerin bellekte farklı yerlerde durup ayrı kimliğe sahip olması doğrudur.

Sakıncalı olabilecek bir durumu hoca kalıptan bağımsız bir örnekle anlattı: alışveriş firmaları "5 milyon müşterimiz var" gibi istatistikler açıklar. Burada tekil kişiler mi sayıldı, yoksa bir kişi 10 sipariş verdiğinde 10 müşteri mi sayıldı? Bu soruda müşterinin kimliği önemlidir. Böyle bir sakınca yoksa Flyweight'in de sakıncası yoktur.

5. Flyweight Uygulama Örneği: Kimya Fabrikası

5.1 Problem

Örnek, Design Patterns kitabından alınmıştır. Bir kimya fabrikası için üretim bilgisayar sistemi hazırlanıyor. Fabrikada birçok kimyasal madde üretiliyor ve birkaç kimyasal maddenin birleşiminden oluşan çok sayıda karışım hazırlanıyor. Üretilen bir madde birden fazla karışımda yer alabiliyor.

Hoca burada önemli bir ayrıma dikkat çekti. Son cümle olmasa, "maddelerin birleşiminden karışımlar oluşuyor" ifadesi akla Composite'i getirebilirdi. Ama anahtar sözcük, aynı maddenin birden fazla karışımda yer alabilmesidir; bir madde birçok karışımda ortak kullanılacaktır. Composite'te ise bir varlık belirli bir anda yalnız bir bütünün parçası olabilir, iki ya da üç bütünün parçası olamaz.

5.2 İlk Çözüm

İlk çözümün tasarımı (ekrandan, 00:33:30):

```
KimyasalMadde                                  Karışım
  - gram : double                                - isim : String
  - isim : String                                - maddeler : LinkedList<KimyasalMadde>
  - simge : String                               + setIsim( String )
  - atomAgirligi : double                        + getIsim( ) : String
  + Kimyasal( isim, simge, atomAg : double )     + maddeEkle( KimyasalMadde )
  + setGram( double )                            + tarifEt( ) : String
  + getGram( ) : double
  + getIsim( ) : double
  + getAtomAgirligi( ) : double
  + getMol( ) : double    // return gram/atomAgirligi;

Karışım <>----> KimyasalMadde (sahiplik)
```

Karışımın bir adı vardır ve karışıma giren maddeler bir bağlı listede tutulur. `tarifEt()`, "şu kadar gram şu madde" biçiminde bir metin oluşturup döndürür. `KimyasalMadde`'de maddenin gramajı, adı ve periyodik tablodaki simgesi bulunur; daha gerçekçi bir uygulamada kimya bilgisi gerekeceği için atom ağırlığı da konmuştur. Kurucu, gramaj dışındaki bütün üyeleri alır; gramaj değişebileceği için kurucuya verilmez, diğerleri maddeye ait değişmez bilgilerdir. `getMol()` gram bölü atom ağırlığı olarak kodlanmıştır.

Slayta göre kaynak kod basit olduğu için notlarda verilmemiş, sıkıştırılmış dosyada `dp.flyweight.solution1` paketlerinde verilmiştir: `a` doğru olmayan bu asıl çözüm, `b` ve `c` yine doğru olmayan alt maddelerle ilgili çözümlerdir. Hoca, verimli olmayan bu çözümlerin ayrıntılarına girmeye değmeyeceğini söyledi.

Örnek: 100 g barutta 75 g potasyum nitrat, 15 g karbon ve 10 g kükürt bulunur. Bu çözüm böyle tarifleri modellemeye yeter. Ama karbon başka karışımlarda başka gramajlarla da bulunacaktır; örneğin grafitte çok daha büyük oranda karbon vardır. Slayttaki soru şudur: yalnız gram bilgisi farklı, kalan durum bilgisi aynı olan çok sayıda karbon nesnesi mi oluşturulmalıdır?

5.3 Daha Doğru Modelleme

Hoca yine "neyin değiştiğini bul ve onu sarmala" ilkesini hatırlattı. Karışımlar yine kimyasal maddelerden oluşur, ama her kimyasal madde karışımın bir bileşeni olur. Bu yüzden bir `Bileşen` sınıfı oluşturulur ve bileşenin değişen kısmı olan gram bu sınıfa alınır (ekrandan, 00:40:00):

```
Bileşen                                  Karışım
  - gram : double                          - isim : String
  - madde : KimyasalMadde                  - bileşenler : LinkedList<Bileşen>
  + Bilesen( double, KimyasalMadde )       + setIsim( String )
  + getGram( ) : double                    + getIsim( ) : String
```

```
  + getMol( ) : double                    + bileşenEkle( madde : KimyasalMadde, gram : double
  + getMaddeAdi( ) : String               + tarifEt( ) : String
      | 1                                   (not: veya bileşenEkle( Bileşen ))
      v 1
KimyasalMadde
  - isim : String
  - simge : String
  - atomAgirligi : double
  + Kimyasal( isim : String, simge : String, atomAg : double )
  + getIsim( ) : double
  + getAtomAgirligi( ) : double
  + getMol( gram : double ) : double
```

Bir bileşen tek bir kimyasal maddeyle ilişkilidir. Maddenin adı, atom ağırlığı ve mol bilgisi bileşenden de istenebilir; bu yüzden `Bileşen`'deki `getMol()` ve `getMaddeAdi()` köprü ya da delegasyon metotlarıdır. Örneğin `getMol()`'ün gövdesi `madde.getMol(...)` çağrısını döndürür. Mol hesabı için gram bilgisi de gerekir ve o artık bileşendedir; bu yüzden bileşen kendi gramını `KimyasalMadde.getMol(gram)` metoduna parametre olarak verir. Hoca, bu ayrıntının kimya bilgisi için değil, delegasyon metotlarının olabileceğini göstermek için konduğunu söyledi.

Bu noktada önemli bir şey fark edilir: daha önce tek bir sınıfla simgelenen kimyasal madde şimdi iki sınıfa ayrılmıştır. Aslında ikisi tek bir varlığı simgeler; varlığın durum bilgisinin bir kısmı (gram) ayrılmış ve farklı farklı kullanılacak, diğer kısmı ortak kullanılacaktır. Yani durum bilgisi içsel ve dışsal olarak ikiye ayrılabilmiştir. Slayttaki sonuç: gram dışsal (extrinsic) bilgi olarak dikkati çeker; çok sayıda madde nesnesi olacağı da düşünülürse Flyweight kalıbına yönelinir.

5.4 Flyweight Kalıbı ile Çözüm

Üçüncü çözümün tasarımı (ekrandan, 00:41:35), ikincisine ek olarak bir `MaddeFabrikasi` sınıfı içerir:

```
MaddeFabrikasi
  - maddeler : HashMap<String, KimyasalMadde> { static }
  + maddeBul( simge : String ) : KimyasalMadde { static }

Karışım
  + bileşenEkle( simge : String, gram : double )   // madde yerine simge
Karışım - - -> MaddeFabrikasi (kullanma ilişkisi)
```

Hoca, birinci ve ikinci çözümde de karışımlara eklenecek madde nesnelerinin bir yerde oluşturulması gerektiğini, bütünsel bir uygulamada bu bilgilerin zaten bir tabloda tutulacağını söyledi. Havuzu idame eden ve ilgili nesneyi döndüren metotlar ya zaten orada olacak ya da küçük bir değişiklikle kalıbın istediği hale getirilebilecektir. Bu yüzden yeni sınıfın getirdiği bağlaşımın sakıncası yoktur; üstelik bu bir sahiplik değil, kullanma ilişkisidir.

Fabrikanın metodu `static` yapılmıştır, çünkü farklı fabrikalar olmayacaktır: bütün karışımların bileşenlerinin maddeleri aynı fabrikadan gelir. Değişen tek şey, karışıma bileşen eklerken maddenin kendisi yerine anahtar alanın, yani periyodik tablodaki simgenin verilmesidir. Hoca, farklı izotoplar kullanılacaksa bu bilginin de ada eklenebileceğini söyledi. `Bileşen` ve `KimyasalMadde` kodlarında hiçbir fark olmamıştır.

`MaddeFabrikasi`'nin kodu (ekrandan, 00:47:45):

```java
package dp.flyweight.solution3;
import java.util.*;
public class MaddeFabrikasi {
    private static HashMap<String, KimyasalMadde> maddeler = new
        HashMap<String, KimyasalMadde>();
    static {
        maddeler.put( "C", new KimyasalMadde("Karbon", "C", 12.0107) );
        maddeler.put( "S", new KimyasalMadde("Kükürt", "S", 32.066) );
        maddeler.put( "KNO3", new KimyasalMadde("Potasyum Nitrat", "KNO3",
```

```
            101.103) );
        maddeler.put( "NaNO3", new KimyasalMadde("Sodyum Nitrat", "NaNO3",
            84.9947) );
    }
    public static KimyasalMadde maddeBul( String simge ) {
        return maddeler.get(simge);
    }
}
```

Hoca bu kodda bir kodlama biçimine dikkat çekti. Bir map'i ya da başka bir veri yapısını kullanmadan önce ilklendirmek gerekir; bu, tanımlandığı yerde de yapılabilir. Ama ilklendirirken içine bazı maddeler de konmak isteniyorsa, ilklendirmenin ardından `static { ... }` bloğu içinde veri yapısına eklenecek öğeler yazılabilir. Hoca bu gösterimi bu örnekle öğrendiğini söyledi. Alternatif olarak `public static` bir "ilk maddeleri ekle" metodu yazılabilir, ama o zaman bu metodu çağırmayı unutmamak gerekir; `static` blokta unutmak mümkün değildir. Bu gösterimin desteklenmediği dillerde böyle bir metot kullanılabilir; `static` nesne gerekmiyorsa normal bir fabrika nesnesi de oluşturulabilir.

> **Ek bilgi (derste anlatılmadı):** Java'da `static { ... }` bloğuna statik ilklendirme bloğu (static initializer) denir. Sınıf JVM tarafından ilk kez yüklenip ilklendirildiğinde, statik alan ilklendirmeleriyle birlikte kaynak koddaki sırasıyla bir kez çalışır. Bu yüzden `maddeBul()` ilk kez çağrıldığında havuz zaten doludur.

`Karışım.bileşenEkle()` simgeyi alır, `MaddeFabrikasi`'na "bu simgeyi bul, bana ilgili nesneyi döndür" der, dönen nesneyi gramla birlikte yeni bir bileşene verir ve bileşeni listeye ekler. `tarifEt()` bir for-each döngüsü içinde her bileşenin gramını ve adını yazarak tarifi döndürür.

`MainApp` iki karışım oluşturur (ekrandan, 00:50:00):

```
package dp.flyweight.solution3;
public class MainApp {
    public static void main(String[] args) {
        Karisim karaBarut = new Karisim();
        karaBarut.setIsim("Kara barut");
        karaBarut.maddeEkle("NaNO3", 75);
        karaBarut.maddeEkle("C", 15);
        karaBarut.maddeEkle("S", 10);
        System.out.println( karaBarut.tarifEt() );
        Karisim temizBarut = new Karisim();
        temizBarut.setIsim("Temiz barut");
        temizBarut.maddeEkle("KNO3", 75);
        temizBarut.maddeEkle("C", 15);
        temizBarut.maddeEkle("S", 10);
        System.out.println( temizBarut.tarifEt() );
    }
}
```

"Kara barut" sodyum nitrat, karbon ve kükürtten oluşur. "Temiz barut" ise daha temiz yanan bir barut olarak sodyum nitrat yerine potasyum nitratla yapılır. Hoca kimya konusunda yanlış söylediyse affedilmesini istedi. Programı çalıştırınca kara barut için 75 g sodyum nitrat, 15 g karbon ve 10 g kükürt görüldü.

> **Ders sonrası not:** Slayttaki kodda iki barutta da 75 + 15 + 10 = 100 g vardır. Derste çalıştırılan sürümde ise "Temiz barut" çıktısı 75 g potasyum nitrat, 20 g karbon ve 25 g kükürt gösterdi (toplam 120 g; ekrandan, 02:00:00, aynı çıktı konsolda kalmıştı). Hoca toplamın 100 etmediğini fark edip doğru oranları kitaptan bakarak düzelteceğini söyledi.

5.5 Çözümlerin Karşılaştırılması

Kod olarak bakıldığında ikinci ve üçüncü çözüm arasında fazla fark yoktur. Fark, saklama noktasındadır. İkinci çözümde, her karışımdaki her madde için bileşenin madde üyesi nedeniyle yine aynı sayıda madde nesnesi tutulur. Birinci çözüm aynı maddenin farklı karışımlarda

farklı gramajlarla kullanılmasını ancak çok sayıda madde nesnesi oluşturarak destekler. İkinci çözümde de nesnelerin büyük kısmı boş yere çoğaltılmış olur. En büyük tasarruf Flyweight kalıbıyla elde edilir; her kimyasal madde nesnesi bütün karışımlarda yeniden kullanılır. Şemalar ve kodlar basit olduğu için hoca kodları tek tek karşılaştırmadı.

6. Proxy Kalıbı

6.1 Amaç ve Örnek Sorun

Proxy, vekil demektir. Hoca, çevirmeli ağ (dial-up) günlerinde internet trafiğinden tasarruf etmek için proxy sunucuları kullanıldığını hatırlattı.

Kalıbın amacı, bir nesneye erişimi denetlemek için bu nesnenin yerine geçecek başka bir nesneyi araya koymaktır. Örneğin ekrana bir resim çizmek zaman alıcı olabilir; ya da çevirmeli ağ günlerinde olduğu gibi boyutu büyük resimlerin indirilmesi zaman alır. Bu yüzden resmin geleceği yere çarpı işaretli küçük bir dörtgen ya da küçük bir önizleme (thumbnail) konurdu.

Hoca bunun bugün de kullanıldığını göstermek için bir video sitesini açtı (ekrandan, 01:10:00). Sayfa ve reklamlar yüklenene kadar içi boş gri alanlar, kanal resimlerinin yerinde de yuvarlak yer tutucular gösterildi. Hiçbir şey görmeden beklesek sayfanın geç yüklendiğini düşünüp sıkılabiliriz; dönen bir kum saati yerine bir şeyler göstermek sabırsızlığı törpüler. Hoca bunun psikolojik bir etki olduğunu söyledi. Yavaş bir bilgisayarda bir sunuda hızla sayfa değiştirirken de resimlerin yerine önce çarpılı küçük bir dörtgen görünür, sayfada durulunca resim gelirdi. Bilgisayarlar hızlandıkça bu durum artık daha çok ağ gecikmesi (latency) olan yerlerde görülür.

6.2 Örnek Çözüm: Resim Vekili

Resim beklenirken araya bir `ImageProxy` konur. Resmin kendisi yüklenene kadar `ImageProxy`, belgede ya da web sayfasında onun yerine bir şey gösterir; arka planda (belki bir iş parçacığında) resmin yüklenmesini bekler, yüklenince onu çizdirir.

Burada önemli olan, vekalet edilen asıl nesne ile vekilin eş biçimde kullanılabilmesidir. Belge düzenleyici (istemci) ikisini de aynı biçimde kullanabilmelidir. Biri bir türlü, öbürü başka türlü kullanılırsa şeffaflık kalkar, iş zorlaşır ve istemcinin fazladan bilgiye ihtiyacı olur. Oysa istemci yalnız resmi diğer içeriklerle birlikte görmek ister; “önce şuna bağlan, sonra buna bağlan” bilgisi istemcinin bilmesi gereken bir şey değildir.

Slayttaki çözüm (ekrandan, 01:13:20):

```
DocumentEditor ----> Graphic (soyut)
                       Draw(), GetExtent(), Store(), Load()   (hepsi soyut)
                          ^ (kalıtım)
              +-----------+-----------+
            Image                   ImageProxy  --image--> Image
              Draw()                  Draw()
              GetExtent()             GetExtent()
              Store()                 Store()
              Load()                  Load()
              imageImp                fileName
              extent                  extent

ImageProxy.Draw():
  if (image == 0) {
      image = LoadImage(fileName);
  }
  image->Draw()

ImageProxy.GetExtent():
  if (image == 0) {
      return extent;
  } else {
```

```
    return image->GetExtent();
}
```

Hem `Image` hem `ImageProxy` ortak `Graphic` arayüzünü destekler. Sınıf soyut ve bütün metotları soyut olduğu için bu bir arayüz de olabilir. Belge düzenleyiciye grafik verileceği zaman önce resim `ImageProxy`'ye, sonra `ImageProxy` nesnesi belge düzenleyiciye verilir. `ImageProxy`, `Graphic`'in alt sınıfı olduğu için belge düzenleyici yine bir grafik nesnesi almış olur; daha önce doğrudan `Image`'a gidecekti.

Proxy, vekalet ettiği nesneyle birebir sahiplik ilişkisi içindedir. Hoca, C++'ta sıfırın `null` anlamına geldiğini hatırlattı: resim henüz yüklenmemişse yüklemeye başlanır, yükleme sürüyorsa arka planda beklenir, sonunda `image->Draw()` (Java'da `image.draw()`) ile resim çizilir. `GetExtent()`'te resim yüklenmemişse varsayılan değer döndürülür; yüklenmişse resmin ilgili metodu çağrılarak delegasyon yapılır.

6.3 Kalıp Yapısı

Kalıp yapısında istemcinin muhatap olduğu `Subject` arayüzü (ya da soyut sınıfı) ve onun iki somut alt sınıfı vardır: `Proxy` ve `RealSubject`. `Proxy`, istemciden gelen `Request()` çağrısını somut olarak kodlarken asıl iş olarak `RealSubject`'in `Request()`'ini çağırır. Öncesinde ya da sonrasında ek işler yapabilir: nesnenin yüklenip yüklenmediğine bakmak gibi. Kalıbın yeteneği, nesneye erişimi denetlemek ve erişimin öncesinde ya da sonrasında ekstra işler yapabilmektir. Çalışma anında istemci, `RealSubject`'in vekiliyle çalışır; ama ortak erişim arayüzü kullanıldığı için bu istemciye şeffaftır, ayrıntılar vekildedir.

6.4 Kullanılabileceği Anlar

Kalıp, araya bir denetim mekanizması koymak gerektiğinde ya da daha genel olarak bir nesneye basit bir işaretçiden daha fazlası gerektiği her durumda kullanılabilir. Hoca şu örnekleri saydı: erişimi denetlemek, gecikmeyi azaltmak ya da daha az hissettirmek, bir nesneye erişmeden önce ödeme yapılıp yapılmadığını kontrol etmek, kullanıcının o hizmete erişim yetkisi olup olmadığını kontrol etmek. Bunlar gibi akla gelen her ekstra yetenek `Proxy` sınıfına kodlanır.

> **Ek bilgi (derste anlatılmadı):** GoF kitabı Proxy'nin başlıca türlerini şöyle sayar: remote proxy (başka adres uzayındaki nesneyi yerel olarak temsil eder), virtual proxy (pahalı nesneyi gerektiğinde oluşturur; resim örneği budur), protection proxy (erişim yetkisini denetler; dersteki ödeme örneği slaytta bu adla anıldı) ve smart reference (erişimde referans sayma gibi ek işler yapar).

7. Proxy Uygulama Örneği: İzlediğin Kadar Öde Sunucusu

7.1 Problem

Slayttaki problem şudur (ekrandan, 01:20:00): izlediğin kadar öde sunucusuna gelen taleplerin emniyet altına alınması.

- Sunucu kendisine verilen adrese istenen programı gönderecek.
- Ancak göndermeden önce istemcinin ödeme yapıp yapmadığına bakılmalı.
- Sunucu yazılımında bu yetenek yok ve yazılımın kaynak koduna erişemiyoruz.
- Araya koyacağımız bir vekil bu işi çözer (protection proxy).

Hoca bunu otellerden bir örnekle anlattı: daha büyük otellerde televizyondan otel içi eğlence sistemine bağlanılabilir, ama paket dışındaki yeni filmler gibi içerikler ayrıca ücretlendirilir ve oda hesabına yansıtılır. Elimizde içerikleri bir yere aktaran bir yayın altyapısı varsa, ama paket dışı filmlerin ücretlendirilmesi gerekiyorsa, bütün sistemi değiştirmeye gerek yoktur. Yayın sunucusunda ücretlendirme modülü yoktur, satın almak daha fazla masraftır; yazılım geliştirme yeteneğimiz varsa bu işi araya konan bir vekille kendimiz çözebiliriz. Vekil, harici ödeme ve faturalama sistemleriyle iletişim işlerini de üstlenir.

7.2 Çözüm

Elimizde zaten bir `StreamServer` vardır: verilen bir programı (program kimliği) verilen bir hedefe gönderir. Hedef, yayının yapılacağı yerdir: bir MAC adresi, bir kullanıcı hesabı ya da bir IP adresi; altyapıda nasıl tutuluyorsa. Müşterinin elindeki TV kutusu `StreamDestination` sınıfıyla simgelenir; müşteri kumandayla "kabul ediyorum, faturama yansıtılmasını kabul ediyorum" gibi işler yapar.

Slayttaki çözüm (ekrandan, 01:23:15) metne şöyle dökülebilir. Hoca var olan kısımları turuncu, yeni eklenenleri yeşil ile işaretledi:

```
Var olan:
  StreamDestination
    - server : StreamSource
    - address : String
    + StreamDestination( address : String )
    + getAddress( ) : String
    + setServer( StreamSource )
    + requestProgram( programID : String ) : boolean
  «interface» StreamSource
    + stream( programID, destination : String ) : boolean
  StreamServer  (StreamSource'u gerçekler)
    + stream( programID, destination : String ) : boolean

Yeni eklenen:
  StreamDestination + makePayment( double ) : boolean
  StreamProxy  (StreamSource'u gerçekler)
    - original : StreamSource
    + StreamProxy( original : StreamSource )
    + stream( programID, destination : String ) : boolean
    + checkPayment( programID, destination : String ) : boolean

İlişkiler:
  StreamDestination 1 ----> 1 StreamSource
  StreamProxy       1 ----> 1 StreamSource (original)
  MainApp ----> StreamDestination, StreamServer, StreamProxy
```

`StreamSource` arayüzü büyük olasılıkla zaten vardı; yoksa oluşturmak da dert değildir. Yeni eklenenler `makePayment()` ile `StreamProxy` sınıfıdır ve `StreamProxy` de arayüz gerçekleme yoluyla sisteme bağlanır. Bundan sonra müşteriler `StreamSource` arayüzü üzerinden `StreamServer` ile değil `StreamProxy` ile iletişimde olur. Yayın yapılmadan önce birinci işlem olarak ödeme kontrolü yapılır; ödeme başarılıysa ikinci olarak yayın işlemi yürütülür. Bu mantık `StreamProxy.stream()` içinde kodlanır: önce `checkPayment()` yapılır, başarılıysa sahiplik ilişkisindeki `original` nesnenin `stream()` metodu çağrılır.

Kaynak kodda `StreamSource` arayüzü "şu programı şu adrese gönder" metodunu tanımlar. `StreamServer` yalnız `stream()` metoduna sahiptir: programı hazırlayıp gönderir, ne ödeme ne başka bir kontrol vardır. `StreamDestination`'a ödemeyi başlatacak metodun eklenmesi gerekir.

`MainApp`'te bir gerçek sunucu ve bir proxy sunucu oluşturulur; proxy'nin kurucusuna gerçek sunucu verilir. İstemcinin adresi belirtilir (`"12:aa:34:ff:54"`, ekrandan, 01:30:00) ve istemciye sunucu olarak gerçek sunucu değil proxy verilir. İstemci bir program istediğinde aslında proxy'ye başvurur; proxy de ödeme yapılmışsa gerçek sunucuya başvurur.

Slaytta `StreamProxy`'nin kodu yoktu; hoca bunu derste iskelet olarak ekledi: `checkPayment()` hedef parametresine göre ödeme yapılmışsa `true`, aksi halde `false` döndürecektir. Hoca gerçek ödeme sistemiyle ilgili ayrıntıyı bilmediği için yalnız bir `if` yapısı kuracağını söyledi.

8. Davranışsal Kalıplara Geçiş: Chain of Responsibility

8.1 Amaç

Yapısal kalıplar bitti ve davranışsal kalıplara Chain of Responsibility ile geçildi. Bu kalıbın amacı, birbiriyle ilişkili bir nesneler zincirinin bulunduğu bir durumda kullanılır. Bu yapı tek bir zincir (iplik) gibi de olabilir, ağaç biçiminde bir hiyerarşi de; fark etmez. Yapı zaten vardır; kalıp, var olan bu yapının üstüne önerilir.

Bu nesnelerden biri, aldığı bir metot çağrısını hakkıyla yerine getiremeyebilir. Bu durumda çağrıyı, işi hakkıyla yapabilecek başka bir nesneye iletmesi gereken bir senaryo yazılmak isteniyorsa bu kalıp kullanılır. Hoca "işlenememesi" değil "hakkıyla işlenememesi" dediğini vurguladı: o sınıfta metot vardır, ama daha fazla ya da daha ayrıntılı bilgiye sahip başka bir nesnenin metodu yürütmesi daha mantıklı olabilir. O başka nesne başka türden olabilir de olmayabilir de.

8.2 Örnek: Bağlama Duyarlı Yardım

Kullanıcı bir diyalog kutusundaki bileşenlerden birinin işlevi hakkında yardım almak istiyor. Hoca bunu bir sunu programında göstermeye çalıştı. F1 tuşu ve sağ üstteki soru işareti, o anki bağlama göre yardım getirir: genel düzeyde basınca uygulamanın ana yardım konu başlıkları gelir, "Yazı Tipi" iletişim kutusundayken soru işaretine tıklayınca doğrudan o iletişim kutusunun yardımı gelir.

Eski sürümlerde bir özellik daha vardı: soru işaretine basınca yardım hemen gelmez, fare imleci soru işaretli bir imlece dönüşürdü. Kullanıcı nereye tıklarsa onunla ilgili en özelleşmiş yardım (bazen bir araç ipucu ile) gelirdi. Hiçbir bileşene değil de başlığa tıklanırsa bütün diyalog kutusunun yardımı gelirdi.

İstenen senaryo şudur: her ekranda ve diyalog kutusunda bir yardım yanıtlama metodu vardır. Bir yere tıklandığında en özelleşmiş yardım gelmeli; o düzeyde bir yardım yoksa daha üst düzey yardım gönderilmelidir. Örneğin belirli bir onay kutusu için ayrı bir yardım yazılmamışsa yazı tipi sekmesinin yardımı, o da yoksa diyalog kutusunun yardımı gelir. Kademeli olarak daha az özelleşmiş yardıma çıkma (ya da tersine en özelleşmiş yardıma inme) senaryosu Chain of Responsibility ile modellenir.

Nesneler arasında şöyle bir ilişki vardır: yazdır düğmesi, tamam düğmesi gibi çeşitli GUI bileşenleri yazdırma diyaloğunun içindedir; diyaloglar da ana uygulamanın içindedir. Bu ilişki en özelden en genele doğru gider. Yazdır düğmesinde yardım istendiğinde, düğmenin kendine özel yardım içeriği varsa o gösterilir; yoksa içinde bulunduğu yazdırma diyaloğunun yardım metodu çağrılır. Onda da yoksa uygulamaya gidilir. Slayttaki şema (ekrandan, 01:40:00):

```
aPrintButton --[!canHelp] HandleHelp()--> aPrintDialog --[!canHelp] HandleHelp()--> anApplicat
```

8.3 Çözüm

Slayttaki çözüm maddeleri (ekrandan, 01:40:00):

- Bileşenler mesajın iletimi için ortak bir arayüze sahip olmalıdır.
- Aksi halde if-else/switch-case ile üst bileşenin tipini test edip uygun bir metodunu çağırmak gerekecektir.
- Önerilen çözümde bir nesne sadece o mesajı yanıtlayıp yanıtlayamayacağını bilmek zorundadır.

Ortak arayüz yoksa if-else-if zinciri kullanmak gerekir; bu bir bakım kabusudur ve hoş bir tasarım değildir. Ortak arayüz yoksa uygun hale getirilmelidir.

> **Hocanın vurgusu:** Kendinizi if-else-if zinciri içinde kod yazarken buluyorsanız, bu büyük olasılıkla bir kalıtım ilişkisi kurup çok biçimlilik yeteneği üzerinden işleri yürütmeniz gerektiğinin alarmıdır.

Hoca GUI bileşenleri arasında zaten hazır bir ilişki olduğunu gösterdi: uygulamada widget'lar vardır; diyalog kutuları, düğmeler, radyo düğmeleri, onay kutuları widget'ın alt sınıflarıdır.

Kalıbın buna ek olarak istediği şey, bir `HelpHandler` sınıfı ve bir handler'ın başka bir handler'la zincir ilişkisinde olduğunu modelleyen üye alandır. Böyle bir çerçevede belki `HelpHandler` adıyla olmasa da ortak bir soyut sınıf zaten bulunabilir; varsa o kullanılır, yoksa biz ekleriz.

Bir düğmede bu işlem şöyle kodlanır: düğmenin ilgili yardım dosyası varsa (şemada "has help" diye kastedilen) yardım gösterilir, yoksa soyut üst sınıftan gelen handler üyesinin `HandleHelp()` metodu çağrılır. Hoca şemada bu çağrının büyük `H` ile yazıldığını, üye adı küçük `h` ile başladığı için küçük `h` olması gerektiğini, ama bunun kesin bir hata olmadığını söyledi.

Bu senaryonun anlamlı olması için ilişkili nesneler zinciri zaten bulunmalıdır. İlişkili nesneler birbirini bilmiyorsa (biri ötekinin içinde olduğunu bilmiyorsa), hoca Composite ile Chain of Responsibility birlikte kullanılabileceğini söyledi.

8.4 Kalıp Yapısı

Slayttaki kalıp yapısı (ekrandan, 01:44:45):

```
Client ----> Handler (soyut)  <--successor-- (kendine)
               HandleRequest()
                  ^ (kalıtım)
          +-------+--------+
  ConcreteHandler1   ConcreteHandler2
    HandleRequest()    HandleRequest()

Çalışma anı:
  aClient.aHandler --> aConcreteHandler.successor --> aConcreteHandler.successor
```

Bir işi (`HandleRequest`) yürütebilecek farklı sınıflar varsa, ortak bir hizmet olduğu için zaten bir soyut üst sınıf ya da arayüz altında toplanmaları gerekir. Ama bu eskiden bir arayüz idiyse, mutlaka soyut sınıfa çekilmelidir; çünkü bir nesnenin başka bir nesneyle zincir ilişkisinde olduğunu kodlamak için `successor` üyesine ihtiyaç vardır. "Successor ekle" gibi zincirin kurulması, izlenmesi ve silinmesi işlemlerini alt sınıflarda tekrar tekrar kodlamak yerine soyut sınıfta kodlamak gerekir. Hoca, UML şemasına "zincir yönetimi metotları da olmalı" diye bir not ekleyeceğini söyledi.

Kalıptaki roller:

- `Client`: yanıtlanacak mesajı zincire gönderir.
- `Handler`: yanıtlanacak mesajı tanımlayan arayüzdür ve zincir bağlantısını gerçeklemelidir; gerçeklemezse alt sınıflarda tekrar eden kod yazmak gerekir.
- `ConcreteHandler`: ilgili mesajı ya yanıtlar ya da daha iyi yanıtlayabilecek bir halef biliyorsa çağrıyı ona iletir.

8.5 Uygulanabileceği Anlar, Zayıf Yönler ve Gerçekleme

Slayttaki maddeler (ekrandan, 01:50:00):

- Kalıbın uygulanabileceği anlar:
  - Bir mesajı yanıtlayabilecek birden fazla nesnenin bulunduğu ve yanıt verecek nesnenin önceden bilinemeyeceği anlarda. Hoca, bilinse bile çalışma anında değiştirilmek istenebileceğini ekledi.
  - Bir mesajı yanıtlayabilecek nesnelerin çalışma anında tanımlanabileceği anlarda.
- Kalıbın zayıf yönleri:
  - Zincir doğru yapılandırılmazsa, bir mesajın yanıtlanacağının garantisi bulunmaz.
- Gerçekleme ayrıntıları:
  - Halihazırda bir zincir yoksa, zinciri oluşturacak metot(lar) `Handler` soyut sınıfında veya `ConcreteHandler` gerçeklemelerinde ayrıca kodlanır.
  - İsteğin birden fazla türden olabileceği durumlarda ya birden fazla `HandleRequestX` metodu tanımlanır ya da isteği simgeleyen bir kalıtım hiyerarşisi kurulabilir.

Hoca zayıf yönü bir devlet dairesi benzetmesiyle anlattı. Bir işin nasıl yapılacağını memura soruyorsunuz; memur "bu konuda yetkim yok, şefime sorayım" diyor. Şef "benim de yetkim yok, şube müdürüne sorayım" diyor; şube müdürü birim müdürüne, birim müdürü "bunun için

bakanlığa başvurmak lazım” diyor. Sonunda herkes “bu konuda yetkimiz yok” der ve iş orada kalır. Zincir doğru kurulmazsa mesaj da böyle yanıtsız kalabilir.

Ders sırasında bir öğrenci Composite ile bu kalıbın farkını sordu; soru ve yanıt “Örnek Sorular” bölümünde verilmiştir.

9. Chain of Responsibility Uygulama Örneği: Reklam Yönetim Yazılımı

9.1 Problem

Örneği hoca kendisi kurgulamıştır (slaytta kaynak olarak kendi baş harfleri yazılıdır). Slayttaki problem (ekrandan, 02:04:50):

- Bir reklam yönetim yazılımı hazırlanıyor.
  - Farklı web sitelerinde kampanyaların çeşitli kurallarına göre çeşitli reklamlar gösterilmek isteniyor.
  - Kampanya 1: Spor sitesinin 18-39 yaş arası erkek ziyaretçilerine tıraş bıçağı reklamı göster, 18-39 yaş arası kadın ziyaretçilerine fitness salonu reklamı göster, diğer ziyaretçilere spor mağazasındaki indirimin reklamını göster.
  - Kampanya 2: Haber sitesinin 18-29 yaş arası A gelir grubu erkek ziyaretçilerine otomobil dergisi reklamını göster, aksi halde spor sitesinin kampanyasını aynen uygula.
  - Kampanya 3: Ziyaretçinin coğrafi bölgesi ve gelir düzeyine göre portföydeki lokanta reklamlarından birini göster.
  - …

Hoca kuralların Chain of Responsibility’yi akla getirecek biçimde yazıldığını, yoksa bu tür problemlerin başka kalıplarla da düşünülebileceğini söyledi. Kampanya 3’ün diğerleriyle ilgisi yoktur, ama kendi içinde bir anlam ilişkisi vardır: önce kullanıcının konumuna bakılır. Konum paylaşılmamışsa farklı illerde şubesi olan bir lokanta zincirinin reklamı gösterilebilir; konuma erişilebiliyorsa gelir düzeyine de bakılır. Hoca gelir düzeyinin nasıl tahmin edildiğini reklamcıların daha iyi bildiğini, ziyaret edilen sitelerden (pahalı arabalar, mücevherler gibi) çıkarım yapılabildiğini söyledi.

Kurgu şudur: birbiriyle ilişkili bir reklam havuzu var; müşteri bu havuzları barındıran sitelerden birine girdiğinde, onu en uygun reklama yönlendirmek isteniyor. Bu düşünce Chain of Responsibility’yi akla getirir.

9.2 Tasarım

`User` sınıfında cinsiyet, gelir düzeyi ve yaş alanları vardır. Cinsiyet ve gelir düzeyi enum olarak tanımlanmıştır; basit kurucu ve getter/setter metotları vardır. Yaşın belirli bir aralıkta olup olmadığını kontrol etmek için bir metot da eklenmiştir; hoca bu işi bilginin en çok bulunduğu sınıfa koymayı tercih ettiğini söyledi.

Soyut `Reklam` sınıfının göstereceği bir resim (`Banner`) vardır; bu kodlanmamıştır. Bir müşteriye gösterilebilecek olası reklamların zinciri, `sonraki` üye alanıyla, sınıfın kendisiyle birebir sahiplik ilişkisi biçiminde tutulur. Zinciri oluşturmak için ayrı bir “sonrakini ata” metodu yazılmamıştır; zincir kurucuda kurulur. Bir reklamın sonraki reklamı yoksa `null` verilir. Zincirin sonundaki reklam, uygunluk metodunda mutlaka `true` döndürerek “ben her kullanıcıya gösterilebilirim” der.

> **Hocanın vurgusu:** Zinciri bir noktada mutlaka sonlandırmanız gerekir.

Bir reklamın bir kullanıcıya uygun olup olmadığına ancak somut reklam sınıfları karar verebilir; bu bilgi soyut sınıfta değil alt sınıflardadır. Alt sınıfların adları ne reklamı olduklarını gösterir: `TrasBicak`, `FitnessSalonu`, `IndirimSporMagaza`, `OtoDergi`.

9.3 Kaynak Kod

Hoca kodu Eclipse’te gösterdi. `Cinsiyet` enum’u (ekrandan, 02:00:00):

```
package dp.chainOfResponsibility.example;

public enum Cinsiyet {
```

```
    KADIN("Kadın"), ERKEK("Erkek");
    private String tanim;
    private Cinsiyet(String tanim) { this.tanim = tanim; }
    public String toString( ) { return tanim; }
}
```

Hoca burada Java'nın nesnel (non-primitive) enum biçimini kullandığını açıkladı. Yalnız `KADIN` ve `ERKEK` de yazılabilirdi; ama ekranda daha ayrıntılı gösterim gerekiyorsa, parantez içindeki değer aslında bir kurucu çağrısıdır. Enum'lar biraz sınıf gibidir: onlardan nesne oluşturulabilir, üye alanlar ve metotlar kodlanabilir. Parantez içindeki metinde Türkçe karakter kullanılabilir, sabit adında kullanılamazdı. Enum kurucusu `private` olur. Tanımın geri döndürülmesi için bir getter yerine `toString()` override edilmiştir. `GelirDuzeyi` ise tamamen yalın bir enum'dur (`A`, `B`, `C`); hoca `Cinsiyet`'i bu yeteneği göstermek için öyle yazmış olabileceğini söyledi.

Soyut `Reklam` sınıfı (ekrandan, 02:02:00):

```
public abstract class Reklam {
    private Banner banner;
    private Reklam sonraki;
    public Reklam(Banner banner, Reklam sonraki) {
        this.banner = banner; this.sonraki = sonraki;
    }
    public abstract boolean reklamUygunMu ( User u );
    public abstract void reklamGoster( User u );
    public final void reklamKotar( User u ) {
        if( reklamUygunMu(u) ) {
            System.out.println( this.getClass().getName()
                + "->" + u );
            reklamGoster(u);
        }
        else if( sonraki != null ) {
            System.out.println( this.getClass().getName()
                + "->" + sonraki.getClass().getName());
            sonraki.reklamKotar(u);
        }
    }
}
```

`reklamKotar()` şöyle çalışır: reklama bir kullanıcı verilir. Reklam bu kullanıcıya uygunsa, kendi sınıf adını ve kullanıcının bilgisini yazar ("bu kullanıcıya kendimi yönlendirdim") ve `reklamGoster()`'i çağırır. Gerçek bir uygulamada resim yüklenip ekrana basılırdı; hoca bu soyut metotların gövdelerini alt sınıflarda boş bıraktığını söyledi. Reklam uygun değilse ve bir sonraki reklam varsa, "devreye girdim ama yönlendirmeyi sonraki reklama yaptım" anlamında kendi adını ve sonrakinin adını yazar ve sonrakinin `reklamKotar()`'ını çağırır. Uygun değilse ve sonraki de yoksa hiçbir şey yapılmaz, reklam gösterilmez.

Zincirin sonundaki `IndirimSporMagaza`, uygunluk metodunda her kullanıcı için `true` döndürür. `reklamKotar()` kalıtımla otomatik olarak gelir. Böylece zincirin sonu olacak bir reklam istendiğinde, "herkese `true` döndür" diyen bir reklamla iş basitçe çözülür.

`OtoDergi` reklamında uygunluk koşulu: cinsiyet erkek, gelir düzeyi A ve yaş 18-29 arasındaysa `true`. Hoca bunun iki `if` ile yazıldığını, `&&` ile de yazılabileceğini, mantığın aynı olduğunu söyledi.

Hoca görev ayrımına dikkat çekti: ana programcı bu altyapıyı kurar. Daha sonra reklam kampanyasının ayrıntısını değiştirmek isteyen uzman bir kullanıcı, ana kodlara hiç dokunmadan `Reklam`'dan kalıtımla yeni bir reklam sınıfı oluşturur. Kurucuyu eklemek kolaydır; reklam göstermede ortak bir iş olsaydı o da üst sınıfa çekilirdi. Uzman kullanıcının dikkat etmesi gereken tek nokta, reklam uygunluğunu doğru yapılandırmaktır.

9.4 Kampanya 1'in Çalıştırılması

`MainApp`'te üç kullanıcı tanımlanmıştır: erkek, C gelir düzeyi, 23 yaş; erkek, A gelir düzeyi, 53 yaş; kadın, B gelir düzeyi, 33 yaş. Reklamlar bir dizide tutulur; aslında önemli olan zincirin ucunu tutmaktır. Zincirin ucunu işaretleyen nesne üzerinden "bu kullanıcıya hangi reklamı göstereceksin" diye işler yürür. Hoca zincirin kökünü dizinin ilk elemanında tutmayı tercih etti. `kampanya1()` (ekrandan, 02:10:00):

```
public void kampanya1( ) {
    Reklam[] reklamlar = new Reklam[3];
    reklamlar[2] = new IndirimSporMagaza(new Banner(), null);
    reklamlar[1] = new FitnessSalonu(new Banner(), reklamlar[2]);
    reklamlar[0] = new TrasBicak(new Banner(), reklamlar[1]);
    reklamlar[0].reklamKotar(users[0]);
    reklamlar[0].reklamKotar(users[1]);
    reklamlar[0].reklamKotar(users[2]);
}
```

Zincir sondan başa kurulur: önce hiçbir şeye bağımlı olmayan, sonraki elemanı olmayan `IndirimSporMagaza` oluşturulur; o `FitnessSalonu`'na, `FitnessSalonu` da `TrasBicak`'a verilir. Zincirin başında `TrasBicak` vardır, çünkü kampanyada ilk bakılacak kural odur.

Konsol çıktısı (ekrandan, 02:10:00):

```
dp.chainOfResponsibility.example.TrasBicak->Erkek,C,23
dp.chainOfResponsibility.example.TrasBicak->dp.chainOfResponsibility.example.FitnessSalonu
dp.chainOfResponsibility.example.FitnessSalonu->dp.chainOfResponsibility.example.IndirimSporM.
dp.chainOfResponsibility.example.IndirimSporMagaza->Erkek,A,53
dp.chainOfResponsibility.example.TrasBicak->dp.chainOfResponsibility.example.FitnessSalonu
dp.chainOfResponsibility.example.FitnessSalonu->Kadın,B,33
```

Hoca çıktıyı adım adım senaryoyla karşılaştırdı:

1. Erkek, C, 23: tıraş bıçağı reklamı devreye girdi ve kendini gösterdi. Senaryoda erkek ve yaş uygun, gelir grubu belirtilmemiş; yani doğru.
2. Erkek, A, 53: yaş uygun olmadığı için tıraş bıçağı reklamı kendini göstermedi, kullanıcıyı fitness salonu reklamına iletti. Fitness salonu kadın kullanıcılar için olduğundan o da göstermedi ve indirimli spor mağazası reklamına iletti. Zincirin sonundaki bu reklam herkesi kabul ettiği için kendini gösterdi.
3. Kadın, B, 33: tıraş bıçağı bir erkek ürünü olduğu için kullanıcıyı fitness salonuna yönlendirdi. Fitness salonu, 33 yaşında kadın kullanıcı şarta uyduğu için kendini gösterdi; boşuna iletmedi.

   Ders sonrası not: Hoca çıktıyı açıklarken (02:11:36) fitness salonu için "18-33 yaş arası" dedi; slayttaki kampanya tanımında aralık 18-39'dur.

Böylece kampanya 1 kapsamında üç farklı kişiye üç farklı reklamın doğru gösterildiği görüldü.

9.5 Kampanya 2'nin Çalıştırılması

Kampanya 2'de dört elemanlı bir dizi kullanılır ve zincirin başına `OtoDergi` eklenir: `OtoDergi` → `TrasBicak` → `FitnessSalonu` → `IndirimSporMagaza`. Hoca, kampanyaları bir bağlı listede tutup zincirin kökünü döndüren bir yapı kursaydı, kampanya 2'nin kampanya 1'i içermesini daha az tekrarlı kodla yazabileceğini söyledi ve bunu öğrencilere bıraktı. Kampanya 3 ise kodlanmadı.

İlk çalıştırmada var olan kullanıcıların hiçbiri otomobil dergisi kuralına uymadığı için `OtoDergi` kendini kimseye göstermedi. Hoca bunu da görmek için 28 yaşında A gelir grubundan bir erkek kullanıcı ekledi. 29 değil 28 seçti ve zincirin başında `OtoDergi` olduğu sürece bu kullanıcının tıraş bıçağı reklamına kaymayacağını belirtti. Zincirin sırası kendi içinde mantıklı olmalıdır: gereksinim "bu yaş ve gelirdeysen otomobil dergisini göster, aksi halde geri kalanı aynen uygula" diye ifade edildiği için `OtoDergi` başta durur. Dizi boyutunu dörde düzeltip yeniden çalıştırınca, `OtoDergi` C grubundaki erkeğe kendini göstermedi, A grubundaki 28 yaşındaki erkeğe gösterdi.

10. Command Kalıbı

10.1 Amaç

Hoca, öğrencilerin onayıyla ara vermeden tek örnekli bir kalıp daha işledi. Command kalıbının amacı, nesnelere giden istekleri, yani metot çağrılarını, bir nesne haline çevirmektir. Bu sayede metot çağrılarını kuyruklamak ya da günlüğünü (log) tutmak gibi işlemler nispeten daha kolay yapılır. Hoca, bunların kalıp olmadan da yapılabileceğini, özellikle günlük tutmanın kolay olduğunu; kuyruklamanın ise çok iş parçacıklı programlama gerektirebileceği için daha zor olacağını söyledi. Kalıp, bunu tamamen ortadan kaldırmayı garanti etmez.

GUI programı yazan herkes bu kalıbı zaten kullanmıştır: bir menü öğesine ya da düğmeye tıklandığında bir nesnenin bir metodunu çalıştıran altyapılar böyledir. Çerçeveler, herhangi bir nesnenin herhangi bir metodunu çağırmak yerine belirli bir arayüzün bir metodunun çağrılmasını sağlar; o arayüzü gerçekleyen bir nesne verildiğinde olay işleme sırasında o metot çağrılır. Bunun arkasındaki tasarım Command kalıbıdır.

10.2 Örnek: Menü Komutları

Bir uygulamanın menüsü ve menünün içinde menü öğeleri vardır. Her menü öğesine tıklandığında, ortak bir komut arayüzünü gerçekleyen farklı komutlardan biri işlenir. Örneğin “Dosya > Aç” tıklandığında açma komutu yürütülür. Kalıpta her komutun başlangıç noktası `Execute()` metodudur.

Slayttaki çözüm (ekrandan, 02:20:00): bir menü seçeneği (`MenuItem` nesnesi) ile bir dosya açma komutu ilişkilendirilebilir.

```
Command (soyut)
  Execute()
     ^ (kalıtım)
  OpenCommand  --application--> Application
    Execute()                     Add(Document)
    AskUser()

OpenCommand.Execute():
  name = AskUser()
  doc = new Document(name)
  application->Add(doc)
  doc->Open()
```

`Execute()` kullanıcıya dosyanın adını sorar, belgeyi oluşturur, açık belgeler arasına ekler ve belgenin `Open()` metodunu çağırarak açılmasını sağlar. Hoca bu varyasyonun, `Application`’dan komuta doğru dolaylı bir bağlantı varken bir somut komutun da `Application` türünden bir üye alanla ona bağımlı olabileceğini göstermek için konduğunu söyledi. Bir başka varyasyonda uygulama içinde birden fazla belge olabilir; bir somut komut, uygulamanın sahip olduğu belgelerle sahiplik ilişkisi kurup örneğin yüklenmiş belgelerin bir metodunu çağırabilir.

10.3 Kalıp Yapısı

Kalıp yapısında soyut bir `Command` sınıfı ve onun soyut `Execute()` metodu vardır. Bu yüzden bu sınıf bir arayüz de olabilir; başka şeylerle ilişkili olması gerekiyorsa soyut sınıf olur. İlk örnekte komutun başka bir yere bağımlılığı yoktur.

- `Command`: komut tanımlama arayüzü.
- `ConcreteCommand`: somut komut gerçeklemeleri. Yürütülecek işler `Execute()` içine kodlanır.
- `Receiver`: komutun yaptığı işten etkilenecek nesne. Önceden bir metotla eklenmiş olabilir ya da somut komut içinde ilgili kod yazılarak yönetilebilir.
- `Client`: somut komut nesnesini oluşturabilir ya da hazır komut verilebilir; gerekirse bir komutu doğrudan yürütmek de isteyebilir.
- `Invoker`: komutların yürütülmesini isteyen, bir tür istemcidir; bu hiyerarşinin içinde de olabilir.

Alıcıdan komut nesnesine doğru bir ilişki olması gerekmez. Probleminiz gerektiriyorsa kodlarsınız; ama her ekstra ilişki ekstra bağlaşım demektir ve gerekmiyorsa kurulmaz.

Kalıp, tanımının gerektirdiği her yerde, yani metotların nesne yapılmak istendiği her noktada kullanılabilir.

11. Command Uygulama Örneği: Akıllı Ev Kumandası

11.1 Problem ve Tasarım

Örnek Head First Design Patterns'ten esinlenmiştir; hoca çok beğendiği bir örnek olmadığını, daha iyisini bulamadığını söyledi. Slayttaki problem (ekrandan, 02:26:30): bir akıllı ev yazılımı üretiyoruz. Yazılımda basit bir açma/kapama kumandası ekranı var. Kumanda ekranındaki aç/kapa düğmelerine odaların ışıklarını, televizyonları vb. açma/kapama işlevleri kazandıracağız.

Bir düğmeye tıklandığında bir komutun yürütülmesi istendiği için Command kalıbı akla gelir. Tasarım:

```
«interface» AcKapa
  + ac() : boolean
  + kapa() : boolean
     ^ (gerçekleme)
     +-- AydinlatmaDugmesi  (- isim : String; ac(), kapa(), toString(), AydinlatmaDugmesi(isim
     +-- ElektronikAlet     (- isim : String; ac(), kapa(), ElektronikAlet(isim), toString())
     +-- Iklimlendirme      (- isim : String; ac(), kapa(), Iklimlendirme(isim), toString())

Kumanda  ---- * ----> AcKapa
  - acKapaDugmeleri : ArrayList<AcKapa>
  + ac(dugmeNo : int) : boolean
  + Kumanda()
  + dugmeEkle(dugme : AcKapa)
  + kapa(dugmeNo : int) : boolean
  + hepsiniKapa()

AnaProgram ----> Kumanda
  - kumanda : Kumanda
  + AnaProgram()
  + evBoskenSokakKapisiAcildi()
  + evdeKimseKalmadi()
  + main(args : String[])
```

Kumanda ekranındaki aç/kapa düğmeleri bir listede tutulur. Kumanda ilklendirilebilir, belirli numaralı bir düğmenin `ac()` ya da `kapa()` metodu çağrılabilir. Evden çıkarken her şeyi birlikte kapatmak için bir `hepsiniKapa()` metodu da eklenmiştir; hepsini açmak anlamlı olmadığı için öyle bir metot yoktur. Ana programda bir kumanda bulunur ve akıllılık mantığı buraya kodlanır: "ev boşken sokak kapısı açıldığında şunu yap", "evde kimse kalmadığında şunu yap" gibi küçük senaryolar.

Aç/kapa düğmeleri basit bir arayüzü gerçekler: açma ve kapama metotları işlem başarılıysa `true`, aksi halde `false` döndürür. Asıl iş bu arayüzü gerçekleyen aydınlatma düğmesi, elektronik alet düğmesi ve iklimlendirme cihazı (klima ya da kombi) düğmesi sınıflarındadır.

11.2 Kaynak Kod

`AydinlatmaDugmesi` `AcKapa`'yı gerçekler; kurucusuna "Salon ışıkları" gibi bir ad verilir. Metotlar basitçe `true` döndürür. Hoca, açıkken yeniden açmaya çalışınca `false` döndürmek gibi ayrıntılara kalıbı örneklendirmek için gerek olmadığını söyledi. Diğer düğme sınıfları da hemen hemen aynıdır.

`Kumanda`'da düğmeler bir listede tutulur ve liste kurucuda ilklendirilir. `dugmeEkle()` listeye yeni eleman ekler. `ac(dugmeNo)` önce o numarada bir düğme olup olmadığına bakar; yoksa `false` döndürür, varsa düğmeyi alıp `ac()` metodunu çağırır ve onun sonucunu döndürür (bu örnekte

hep `true`). `kapa()` benzer biçimde kodlanmıştır. `hepsiniKapa()` listedeki bütün nesneleri dolaşıp `kapa()` metodunu çağırır.

> **Ders sonrası not:** Hoca listeyi anlatırken bağlı liste (linked list) dedi; slayttaki sınıf şemasında alan `ArrayList<AcKapa>` olarak görünür (ekrandan, 02:26:30).

`AnaProgram` (ekrandan, 02:30:00):

```
public class AnaProgram {
    Kumanda kumanda;
    public AnaProgram( ) {
        kumanda = new Kumanda();
        kumanda.dugmeEkle( new AydinlatmaDugmesi( "Antre" ) );
        kumanda.dugmeEkle( new AydinlatmaDugmesi( "Salon" ) );
        kumanda.dugmeEkle( new AydinlatmaDugmesi( "Mutfak" ) );
        kumanda.dugmeEkle( new AydinlatmaDugmesi( "Oda" ) );
        kumanda.dugmeEkle( new ElektronikAlet( "Vestel TV" ) );
        kumanda.dugmeEkle( new Iklimlendirme( "Kombi" ) );
    }
    public void evBoskenSokakKapisiAcildi( ) {
        kumanda.ac(0); kumanda.ac(5);
    }
    public void evdeKimseKalmadi( ) { kumanda.hepsiniKapa( ); }
    public static void main(String[] args) {
        AnaProgram prg = new AnaProgram();
        prg.evBoskenSokakKapisiAcildi();
        System.out.println("Evden çıkılıyor");
        prg.evdeKimseKalmadi();
    }
}
```

Ana programın kurucusunda ev donatılır: kumanda oluşturulur ve girişin, salonun, mutfağın ve odanın aydınlatma düğmeleri, televizyon ve kombi kumandaya eklenir. Senaryolarda, ev boşken sokak kapısı açıldığında 0 numaralı düğme (girişin ışığı) ve kumandanın 5 numaralı düğmesi açılır; evde kimse kalmadığında `hepsiniKapa()` çağrılır. `main` senaryoları gösterim için çalıştırır.

> **Ders sonrası not:** Hoca bu senaryoyu "girişin ışığı ve televizyon açılsın, eve girince bizi televizyon karşılasın" diye anlattı. Listede 0 numara "Antre", 4 numara "Vestel TV", 5 numara "Kombi"dir; düğmeler sıfırdan numaralandığı için `kumanda.ac(5)` büyük olasılıkla televizyonu değil kombiyi açar. Hocanın anlattığı senaryo için `kumanda.ac(4)` yazılmalıdır.

Hoca örneğin basit ve iskelet olduğunu, asıl önemli olanın arka plandaki ilişkileri şemada görselleştirip koda aktarabilmek olduğunu söyledi.

Örnek Sorular

Soru 1 (ekrandan, 00:33:30): Kimya fabrikası örneğinin ilk çözümünde `KimyasalMadde` sınıfı `gram`, `isim`, `simge` ve `atomAgirligi` alanlarını tutuyor. 100 g barutta 75 g potasyum nitrat, 15 g karbon ve 10 g kükürt bulunur; karbon başka gramajlarla başka karışımlarda da bulunacaktır. Sadece gram bilgisi farklı olan, kalan durum bilgisi aynı olan çok sayıda karbon nesnesi mi oluşturulmalıdır?

> **Çözüm (derste verildi):** Hayır. Değişen gram bilgisi ayrı bir `Bileşen` sınıfına alınır; `Bileşen` gramı ve bir `KimyasalMadde` referansını tutar. Böylece gram dışsal bilgi, madde adı, simgesi ve atom ağırlığı paylaşılan içsel bilgi olur. Ardından madde nesneleri simgeyle anahtarlanan `static` bir `MaddeFabrikasi` havuzundan alınır (Flyweight); her madde bütün karışımlarda tek bir nesneyle temsil edilir.

Soru 2 (derste sözlü, 00:31:42): Kimya örneğinde maddeler birleşerek karışımları oluşturduğuna göre neden Composite değil de Flyweight kullanılıyor?

> **Çözüm (derste verildi):** Anahtar nokta, aynı maddenin birden fazla karışımda ortak kullanılmasıdır. Composite'te bir varlık belirli bir anda yalnız bir bütünün parçası olabilir; iki ya da üç bütünün parçası olamaz. Ortak kullanım ve çok sayıda benzer nesne Flyweight'i gerektirir.

Soru 3 (derste sözlü, 01:15:58): İstemci önceden doğrudan `RealSubject` ile sahiplik ilişkisindeyse, araya bir `Subject` arayüzü koyup kodu buna göre değiştirmek ne kazandırır?

> **Çözüm (derste verildi):** Proxy kalıbının getirdiği erişim denetimine ihtiyaç varsa bu değişiklik yapılır, yoksa kalıp kullanılmaz. Gerekiyorsa, istemcinin `RealSubject`'te gerçekten kullandığı metotlar ayıklanıp (extract) yeni `Subject` arayüzüne konabilir. `Subject` baştan bir arayüz olarak tasarlanmışsa kalıbın kurulması çok daha kolay olur.

Soru 4 (derste sözlü, 01:48:44): Bir öğrenci sordu: Bir problemin birden fazla tasarım kalıbıyla modellenebileceği söylenmişti. Bağlama duyarlı yardım probleminde Composite yerine Chain of Responsibility kullanılmasının sebebi nedir? Öğrenci, bir bileşenin yardımı yoksa bir üst bileşene gitmeyi hiyerarşik bir parça-bütün ilişkisi olarak düşünmüştü.

> **Çözüm (derste verildi):** Composite, hiyerarşik ilişkileri ağaç yapısında modeller: her parçanın da alt parçaları olabilir ve kalıbın amacı parça-bütün ilişkilerini modellemektir. Chain of Responsibility ise hem hiyerarşiyi hem de düz bir zinciri destekler; ağaç biçimli bir yapıda bile en alt yapraktan köke doğru giden bir zincir vardır ve bu kalıp için zincirin olması yeter. Burada amaç, yapılacak bir işin ilişkili bileşenler zinciri içinde en doğru yerinin bulunup yürütülmesidir. Yardım örneğinde de bir zincir izlenerek yalnız yukarı doğru gidilir, aşağılara inilmez.

Soru 5 (ekrandan, 02:10:00): Kampanya 1 zinciri `TrasBicak` → `FitnessSalonu` → `IndirimSporMagaza` olarak kurulmuştur. (Erkek, C, 23), (Erkek, A, 53) ve (Kadın, B, 33) kullanıcılarına hangi reklamlar gösterilir?

> **Çözüm (derste verildi):** (Erkek, C, 23) → `TrasBicak` kendini gösterir (erkek, yaş 18-39 arası). (Erkek, A, 53) → `TrasBicak` yaş uymadığı için iletir, `FitnessSalonu` kadın olmadığı için iletir, `IndirimSporMagaza` herkese uygun olduğu için gösterir. (Kadın, B, 33) → `TrasBicak` iletir, `FitnessSalonu` kendini gösterir.

Dersin Toparlaması

Bu derste yapısal kalıplar Facade, Flyweight ve Proxy ile tamamlandı. Facade, alt sistemlerin dağınık arayüzlerini tek bir üst düzey arayüzde toplar. Flyweight, çok sayıda benzer nesnenin değişmeyen içsel kısmını bir fabrika havuzunda paylaştırır ve değişen dışsal kısmı kullanıldığı yerden metot parametresiyle verir. Kimya fabrikası örneğinde bu, önce değişen gram bilgisinin `Bileşen` sınıfına ayrılmasıyla, sonra maddelerin simgeyle anahtarlanan `static` bir `MaddeFabrikasi` havuzundan alınmasıyla gösterildi. Proxy, asıl nesneyle aynı arayüzü destekleyen bir vekil koyarak erişimi istemciye şeffaf biçimde denetler: resim yer tutucusu ve ödeme kontrolü yapan yayın vekili bunun örnekleridir.

Ardından davranışsal kalıplara geçildi. Chain of Responsibility, bir isteği yanıtlayabilecek nesne bulunana kadar `successor` zinciri boyunca iletir; zincirin doğru kurulması ve bir noktada sonlanması gerekir. Reklam yönetim örneğinde `final` bir `reklamKotar()` metodu ve zincirin sonunda herkese uygun bir reklamla bu sağlandı. Command, metot çağrılarını `Execute()` metoduna sahip nesnelere dönüştürür ve GUI olay altyapılarının temelini oluşturur. Akıllı ev kumandası örneği bunu basit bir `AcKapa` arayüzüyle gösterdi. Hoca bir sonraki derste davranışsal kalıplarla devam edileceğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.